Data Structures

BS (CS) _Fall_2025

Lab_03 Manual

Learning Objectives:

1. 2D and 3D arrays

2D & 3D arrays in C++

Arrays

In C++, an array is a data structure that is used to store multiple values of similar data types in a contiguous memory location.

Basics

Passing arrays to function & Array Decays to Pointer:

```cpp
#include <iostream>
using namespace std;

void printArray(int arr[], int size) {   // arr is actually int*
    for(int i = 0; i < size; i++)
        cout << arr[i] << " ";
    cout << endl;
}

int main() {
    int nums[5] = {10, 20, 30, 40, 50};
    printArray(nums, 5);   // Passed as int*
}
```

When we pass an array to a function, it decays into a pointer to its first element.

So inside the function, the size information is lost.

Reference to an Array:

If we want the function to know the exact size at compile time, we can pass by reference to an array.

```cpp
template <typename T, size_t N>
void printArray(const T (&arr)[N]) {   // reference to array of size N
    cout << "Size = " << N << endl;
    for (size_t i = 0; i < N; i++)
        cout << arr[i] << " ";
    cout << endl;
}

int main() {
    int nums[5] = {1, 2, 3, 4, 5};
    printArray(nums);   // N automatically deduced as 5
```

```
}
```

Here, the function knows size = 5 at compile time because it's part of the type (int[5]).

Dynamic 1D Arrays

Using new and delete:

Unlike static arrays (int arr[10];), the size of a dynamic array can be decided at runtime.

We use **new** to allocate memory and **delete[]** to free it.

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter size: ";
    cin >> n;

    int* arr = new int[n];   // allocate dynamic array

    for(int i=0; i<n; i++)   // fill array
        arr[i] = i * 10;

    for(int i=0; i<n; i++)   // print array
        cout << arr[i] << " ";

    delete[] arr;  // free memory
}
```

Resizing dynamic arrays

```
int* resizeArray(int* oldArr, int oldSize, int newSize) {
    int* newArr = new int[newSize];       // new block
    for(int i=0; i<oldSize; i++)          // copy old elements
        newArr[i] = oldArr[i];
    delete[] oldArr;                      // free old memory
    return newArr;
}
```

If we want a bigger array:

- Allocate a new block.

- Copy old elements.
- Delete the old block.

Dynamic 2D Arrays

When we don't know rows and columns at compile time, we create a 2D array with pointers.

Creating a 2D Array with Pointers

```
int rows = 3, cols = 4;

// step 1: create array of row pointers
int** arr = new int*[rows];

// step 2: create each row
for(int i=0; i<rows; i++)
    arr[i] = new int[cols];
```

Accessing Elements

```
arr[0][0] = 10;     // first row, first column
arr[1][2] = 20;     // second row, third column
```

Internally, arr[i][j] is the same as:

```
*(*(arr+i) + j)
```

Dynamic 3D Arrays (Brief)

When we want an array with 3 dimensions (like [x][y][z]), we allocate step by step:

Creating a 3D Array

```
int x = 2, y = 3, z = 4;

int*** arr = new int**[x];          // x "blocks"
for(int i=0; i<x; i++) {
    arr[i] = new int*[y];           // y "rows" in each block
    for(int j=0; j<y; j++) {
        arr[i][j] = new int[z];     // z "cols" in each row
    }
}
```

Accessing Elements

```
arr[0][1][2] = 50;    // block 0, row 1, col 2
cout << arr[0][1][2]; // prints 50
```

Internally, this is equivalent to:

```
*(*(*(arr+0)+1)+2)
```

arr[i][j][k] → pointer-of-pointer-of-pointer.

Returning Arrays in Templates

We can return a dynamic array (pointer) from a template function.

```
#include <iostream>
using namespace std;

template <typename T>
T* createArray(int size, T initVal) {
    T* arr = new T[size];        // dynamic array
    for(int i=0; i<size; i++)
        arr[i] = initVal;
    return arr;                  // return pointer
}

int main() {
    int* intArr = createArray<int>(5, 10);
    double* dblArr = createArray<double>(3, 2.5);

    for(int i=0; i<5; i++) cout << intArr[i] << " ";  // 10 10 10 10 10
    cout << endl;
    for(int i=0; i<3; i++) cout << dblArr[i] << " ";  // 2.5 2.5 2.5

    delete[] intArr;
    delete[] dblArr; // free memory
}
```